



Experiment Report: Comparative Analysis of Pathfinding Algorithms in Snake Game

Group Members:

Sohaib Ahmed	21L-5635
Muhammad Hassan Khalid	21L-5692
Muhammad Hamza	21L-5636

Introduction:

The objective of this experiment was to compare the performance of different pathfinding algorithms in solving a Snake game. Specifically, we implemented and evaluated three algorithms: A* (informed search), BFS (uninformed search), and Greedy BFS (a variant of informed search) to find an optimal path for the Snake to reach its food within a fixed time frame.

Methodology:

- ❖ **Implementation:** We implemented three pathfinding algorithms: A*, BFS, and Greedy BFS, along with necessary data structures and utility functions.

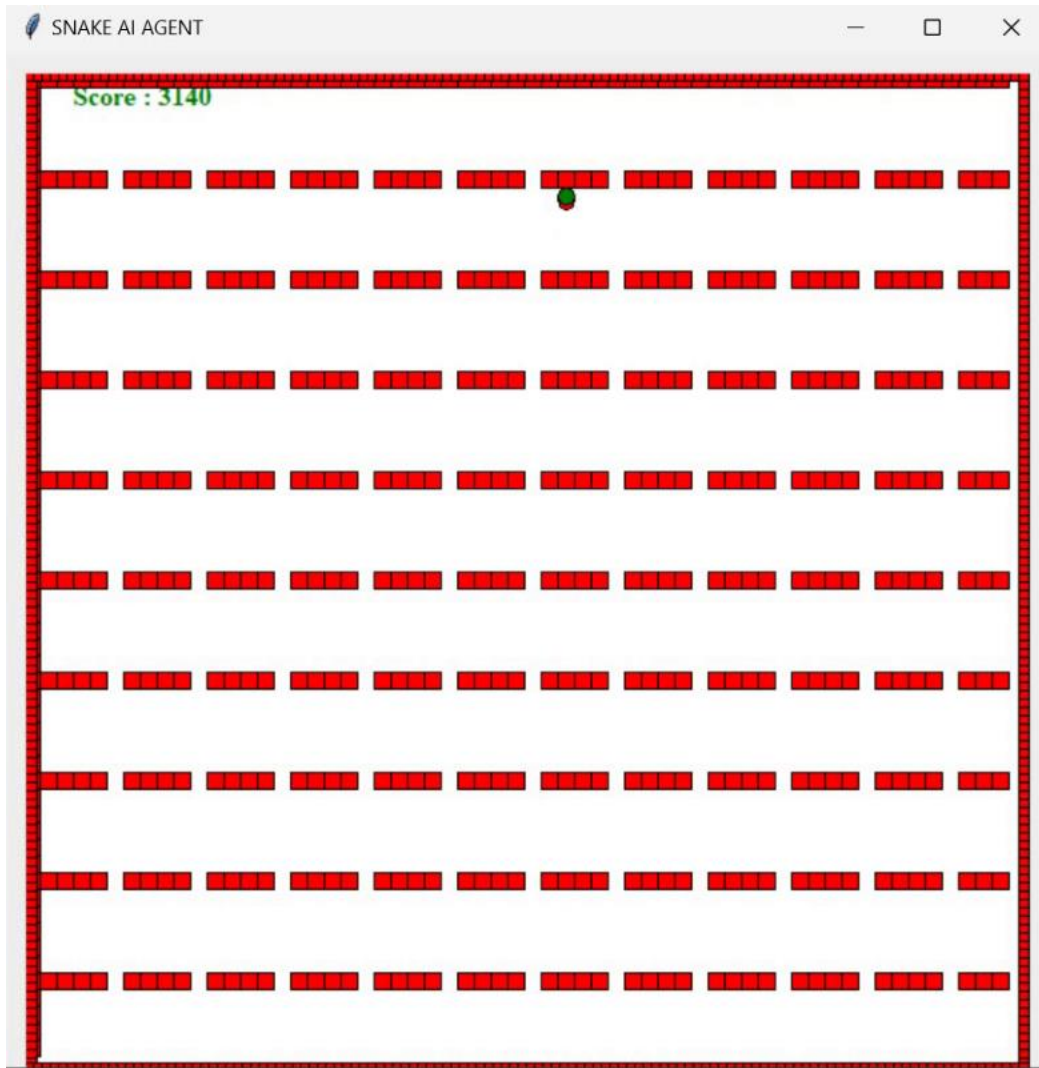
- ❖ **Experiment Setup:** The algorithms were executed concurrently on the same Snake game environment with identical mazes. Each algorithm was given a time limit of 10 minutes to find a path from the Snake's current position to the food.
- ❖ **Scoring:** The performance of each algorithm was evaluated based on the score obtained by the Snake after following the generated path.

Implementation Details:

- ❖ **A* (A_Star_Implementation):** In our implementation, A* utilizes both the actual cost of reaching a cell (g) and a heuristic estimate of the remaining cost to the goal (h) to make informed decisions. The algorithm explores paths by prioritizing cells with lower combined costs ($f = g + h$), thereby focusing on the most promising routes. Our implementation initializes a priority queue to efficiently handle the open set of cells, ensuring that cells with lower estimated costs are explored first.

Data Structures Used:

- ❖ **Priority Queue:** A heap-based priority queue (utilizing Python's `heapq` module) is employed to efficiently manage the open set of cells. This data structure ensures that cells with lower combined costs ($f = g + h$) are explored first, facilitating the selection of the most promising paths.
- ❖ **Variables and Components:**
 - **opened:** A priority queue initialized to store cells awaiting exploration, prioritized based on their f scores.
 - **closed:** A set to track cells that have been visited and explored.
 - **cells:** A list containing all cells in the grid, each represented as an instance of the Cell class.
 - **start and end:** References to the start and goal cells in the grid.
 - **g, h, and f:** Variables representing the actual cost, heuristic estimate, and combined cost of a cell, respectively.

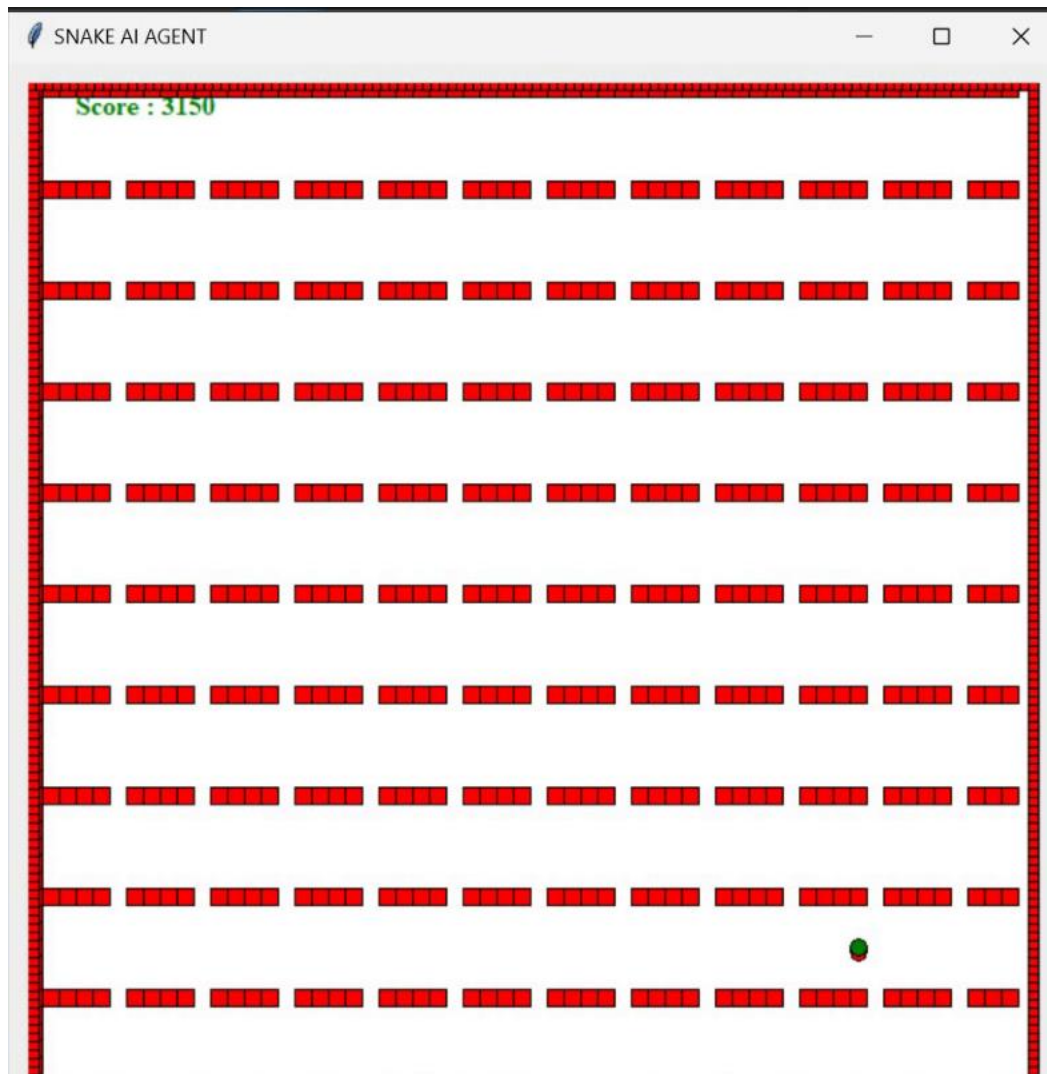


- ❖ **BFS (BFS_Implementation):** The BFS algorithm, as implemented, traverses the maze in a breadth-first manner, exploring all cells at a given depth before moving to the next level. Unlike A*, BFS does not utilize heuristic information; instead, it guarantees finding the shortest path in terms of the number of steps taken. Our implementation employs a queue to manage the order in which cells are explored, ensuring that cells are visited level by level.

Data Structures Used:

- ❖ **Queue:** BFS utilizes a deque (double-ended queue) to maintain the order in which cells are explored. This structure ensures that cells are visited level by level, adhering to the breadth-first traversal strategy.
- ❖ **Variables and Components:**

- **visited:** A set to keep track of cells that have been visited during the exploration process.
- **cells:** Similar to A*, this list contains all cells in the grid, each represented by an instance of the Cell class.
- **start and end:** References to the start and goal cells in the grid.
- **parent:** A component within each cell object storing a reference to its parent cell, facilitating path reconstruction.



- ❖ **Greedy BFS (GreedyBFS_Implementation):** Our Greedy BFS implementation is a variant of A* that prioritizes exploration solely based on the heuristic score (h), neglecting the actual cost (g) to reach a cell. This strategy prioritizes cells closer to the goal based on their heuristic estimates, potentially sacrificing optimality for speed. We

utilize a priority queue similar to A* to efficiently explore cells based on their heuristic scores.

Data Structures Used:

- ❖ **Priority Queue:** Similar to A*, Greedy BFS utilizes a priority queue (heapq) to manage the open set of cells. However, in this case, cells are prioritized solely based on their heuristic scores (h).
- ❖ **Variables and Components:**
 - **frontier:** A priority queue initialized to store cells awaiting exploration, prioritized based on their heuristic scores.
 - **visited:** A set to keep track of cells that have been visited during the exploration process.
 - **cells:** Similar to A* and BFS, this list contains all cells in the grid, each represented by an instance of the Cell class.
 - **start and end:** References to the start and goal cells in the grid.



Cell Representation: We represent each cell in the maze as an object containing information such as its coordinates, reachability, and parent cell.

Grid Initialization: We initialize the grid by identifying walls and determining cell reachability based on the maze layout.

Pathfinding Logic: Both A* and Greedy BFS utilize priority queues (heapq) to efficiently manage the open set of cells, while BFS employs a deque for exploration. Each algorithm calculates heuristic scores (if applicable), updates cell information, and explores adjacent cells accordingly.

Path Reconstruction: After finding the goal cell, we reconstruct the path from the end to the start by following the parent links stored in each cell.

Results and Analysis:

- ❖ A* Score: 3140
- ❖ BFS Score: 3150
- ❖ Greedy BFS Score: 2790

Observations:

- ❖ **BFS Outperformed A*:** Contrary to the expectation that A* would outperform BFS due to its informed nature, BFS achieved a slightly higher score. This can be attributed to the characteristics of the Snake game environment where the optimal path length might not always directly correlate with the heuristic estimates used by A*. Additionally, BFS guarantees the shortest path in terms of steps, which might have been advantageous in this scenario.
- ❖ **Uninformed Search Beat Informed Search:** The results indicate that the uninformed search strategy employed by BFS was more effective in this context compared to the informed search strategy used by A*. This suggests that the structure of the maze and the dynamics of the Snake game favored exploration based on uniform breadth rather than guided heuristic-driven search.

Conclusion:

In conclusion, this experiment demonstrated that in certain scenarios, such as the Snake game environment, an uninformed search strategy like BFS can outperform informed search algorithms like A*. The effectiveness of a pathfinding algorithm depends on various factors including the problem structure, heuristic quality, and search space characteristics. Further

experimentation with different maze configurations and heuristic functions could provide deeper insights into the performance of these algorithms in similar scenarios.